

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №2.

Тема: «Классы и объекты. Использование конструкторов.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

1. Определить пользовательский класс.
2. Определить в классе следующие конструкторы: без параметров, с параметрами, копирования.
3. Определить в классе деструктор.
4. Определить в классе компоненты-функции для просмотра и установки полей данных (селекторы и модификаторы).
5. Написать демонстрационную программу, в которой продемонстрировать все три случая вызова конструктора-копирования, вызов конструктора с параметрами и конструктора без параметров.

Анализ

Класс `Tovar` хранит название, количество и стоимость товара, а также предоставляет конструкторы, деструктор, геттеры, сеттеры и метод `show` для вывода данных. Конструктор без параметров, с параметрами и копирования, а также деструктор выводят сообщения, чтобы отследить моменты их вызова. Функция `func_copу_by_value` принимает объект по значению, что вызывает конструктор копирования. Функция `func_return_object` создаёт локальный объект и возвращает его, что также приводит к копированию. В `main` последовательно демонстрируются все виды конструкторов, работа геттеров и сеттеров, а также три случая вызова конструктора копирования.

Код

```
#include <locale>
#include <iostream>
#include <string>
using namespace std;

class Tovar
{
private:
    string naimenovanie;
    int kolichество;
    double stoimost;

public:
    Tovar();
    Tovar(string, int, double);
    Tovar(const Tovar&);
    ~Tovar();
    string get_naimenovanie();
    int get_kolichество();
```

```

    double get_stoimost();
    void set_naimenovanie(string);
    void set_kolichestvo(int);
    void set_stoimost(double);
    void show();
};

Tovar::Tovar()
{
    naimenovanie = "";
    kolichestvo = 0;
    stoimost = 0.0;
    cout << "Конструктор без параметров" << endl;
}

Tovar::Tovar(string N, int K, double S)
{
    naimenovanie = N;
    kolichestvo = K;
    stoimost = S;
    cout << "Конструктор с параметрами" << endl;
}

Tovar::Tovar(const Tovar& t)
{
    naimenovanie = t.naimenovanie;
    kolichestvo = t.kolichestvo;
    stoimost = t.stoimost;
    cout << "Конструктор копирования" << endl;
}

Tovar::~Tovar()
{
    cout << "Деструктор" << endl;
}

string Tovar::get_naimenovanie()
{
    return naimenovanie;
}

int Tovar::get_kolichestvo()
{
    return kolichestvo;
}

double Tovar::get_stoimost()
{
    return stoimost;
}

void Tovar::set_naimenovanie(string N)
{
    naimenovanie = N;
}

void Tovar::set_kolichestvo(int K)
{
    kolichestvo = K;
}

void Tovar::set_stoimost(double S)
{
    stoimost = S;
}

void Tovar::show()
{
    cout << "наименование: " << naimenovanie << endl;
}

```

```

        cout << "количество: " << kolichество << endl;
        cout << "стоимость: " << stoimost << endl;
    }

    void func_copy_by_value(Tovar t)
    {
        t.show();
    }

    Tovar func_return_object()
    {
        Tovar temp("Возвращаемый товар", 10, 99.99);
        return temp;
    }

    int main()
    {
        setlocale(LC_ALL, "Russian");

        cout << " Конструктор без параметров " << endl;
        Tovar tovar1;
        tovar1.show();
        cout << endl;

        cout << " Конструктор с параметрами " << endl;
        Tovar tovar2("Ноутбук", 15, 45000.50);
        tovar2.show();
        cout << endl;

        cout << " Модификаторы " << endl;
        tovar1.set_naimenovanie("Мышь");
        tovar1.set_kolichество(100);
        tovar1.set_stoimost(1250.75);
        tovar1.show();
        cout << endl;

        cout << " Конструктор копирования " << endl;
        Tovar tovar3(tovar2);
        tovar3.show();
        cout << endl;

        cout << " Конструктор копирования " << endl;
        func_copy_by_value(tovar2);
        cout << endl;

        cout << " Конструктор копирования " << endl;
        Tovar tovar4 = func_return_object();
        tovar4.show();
        cout << endl;

        return 0;
    }

```

UML диаграмма

Tovar
- naimenovanie : string - kolichество : int - stoimost : double
<pre> + Tovar() + Tovar(N : string, K : int, S : double) + Tovar(t : const Tovar&) + ~Tovar() + get_naimenovanie() : string + get_kolichество() : int + get_stoimost() : double + set_naimenovanie(N : string) : void + set_kolichество(K : int) : void + set_stoimost(S : double) : void + show() : void </pre>

Вывод программы

```
Конструктор без параметров
Конструктор без параметров
наименование:
количество: 0
стоимость: 0

Конструктор с параметрами
Конструктор с параметрами
наименование: Ноутбук
количество: 15
стоимость: 45000.5

Модификаторы
наименование: Мышь
количество: 100
стоимость: 1250.75

Конструктор копирования
Конструктор копирования
наименование: Ноутбук
количество: 15
стоимость: 45000.5

Конструктор копирования
Конструктор копирования
наименование: Ноутбук
количество: 15
стоимость: 45000.5
Деструктор

Конструктор копирования
Конструктор с параметрами
наименование: Возвращаемый товар
количество: 10
стоимость: 99.99

Деструктор
Деструктор
Деструктор
Деструктор
```

\

Ответы на вопросы

1. Для чего нужен конструктор?

Ответ: Конструктор нужен для инициализации объекта класса при его создании. Он задаёт начальные значения атрибутов, выделяет ресурсы и обеспечивает корректное состояние объекта.

2. Сколько типов конструкторов существует в C++?

Ответ: В C++ существует три основных типа конструкторов: конструктор без параметров (по умолчанию), конструктор с параметрами и конструктор копирования. Также выделяют конструктор перемещения (начиная с C++11).

3. Для чего используется деструктор? В каких случаях деструктор описывается явно?

Ответ: Деструктор используется для освобождения ресурсов, которые были захвачены объектом за время его жизни (память, файлы, соединения и т.д.). Деструктор описывается явно, если класс управляет динамическими ресурсами (например, выделяет память через `new`) или требует выполнения специальных действий при уничтожении объекта.

4. Для чего используется конструктор без параметров? Конструктор с параметрами? Конструктор копирования?

Ответ: Конструктор без параметров используется для создания объекта без начальных значений или со значениями по умолчанию. Конструктор с параметрами используется для создания объекта с заданными начальными значениями. Конструктор копирования используется для создания нового объекта как копии уже существующего объекта.

5. В каких случаях вызывается конструктор копирования?

Ответ: Конструктор копирования вызывается в следующих случаях: при передаче объекта в функцию по значению, при возврате объекта из функции по значению, при явном копировании одного объекта в другой (например, `ClassName obj2 = obj1`), при помещении объекта в контейнер по значению.

6. Перечислить свойства конструкторов.

Ответ: Свойства конструкторов:

- Имя конструктора совпадает с именем класса.
- Конструктор не имеет возвращаемого значения (даже `void`).
- Может быть перегружен.
- Вызывается автоматически при создании объекта.
- Может иметь параметры по умолчанию.
- Не может быть виртуальным (но может быть виртуальным деструктор).
- Не наследуется.

7. Перечислить свойства деструкторов.

Ответ: Свойства деструкторов:

- Имя деструктора совпадает с именем класса, но с тильдой (~) в начале.
- Не имеет параметров.
- Не имеет возвращаемого значения.
- Не может быть перегружен (только один деструктор на класс).
- Вызывается автоматически при уничтожении объекта.
- Может быть виртуальным (особенно важно для полиморфных классов).

8. К каким атрибутам имеют доступ методы класса?

Ответ: Методы класса имеют доступ ко всем атрибутам класса (как к public, так и к private и protected) своего класса. Также они имеют доступ к атрибутам других объектов того же класса.

9. Что представляет собой указатель this?

Ответ: Указатель this — это скрытый указатель, который передаётся каждому нестатическому методу класса. Он содержит адрес текущего объекта, для которого был вызван метод. С помощью this можно обращаться к атрибутам и методам текущего объекта.

10. Какая разница между методами определёнными внутри класса и вне класса?

Ответ: Методы, определённые внутри класса, по умолчанию считаются встраиваемыми (inline). Методы, определённые вне класса, не являются встраиваемыми по умолчанию (если явно не указать inline). В остальном разницы нет: и те, и другие имеют доступ к атрибутам класса.

11. Какое значение возвращает конструктор?

Ответ: Конструктор не возвращает никакого значения. Он не имеет возвращаемого типа (даже void). Синтаксически считается, что конструктор возвращает созданный объект (но не через return).

12. Какие методы создаются по умолчанию?

Ответ: По умолчанию компилятор может создать: конструктор по умолчанию (без параметров), конструктор копирования, оператор присваивания, деструктор. В C++11 и новее также конструктор перемещения и оператор перемещения. Они создаются только если программист явно не определил свои версии и если это необходимо.

13. Какое значение возвращает деструктор?

Ответ: Деструктор не возвращает никакого значения и не имеет возвращаемого типа.

14. Дано описание класса

```
class Student
{
    string name;
    int group;
public:
    student(string, int);
    student(const student&);
    ~student();
};
```

Какой метод отсутствует в описании класса?

Ответ: Отсутствует конструктор без параметров (по умолчанию). Также заметим, что в описании конструктора `student(string, int)` имя конструктора написано с маленькой буквы "student", а имя класса с большой "Student" — это синтаксическая ошибка (должно совпадать).

15. Какой метод будет вызван при выполнении следующих операторов:

```
student* s;
s = new student;
```

Ответ: Будет вызван конструктор без параметров (по умолчанию). Если он явно не описан, но необходим, компилятор может создать его автоматически (при отсутствии других конструкторов). Однако в данном классе из вопроса 14 есть конструктор с параметрами и конструктор копирования, поэтому конструктор по умолчанию автоматически не создаётся, и код вызовет ошибку компиляции.

16. Какой метод будет вызван при выполнении следующих операторов:

```
student s("Ivanov",20);
```

Ответ: Будет вызван конструктор с параметрами (конструктор, принимающий string и int).

17. Какие методы будут вызваны при выполнении следующих операторов:

```
student s1("Ivanov",20);
```

```
student s2 = s1;
```

Ответ: При создании s1 вызывается конструктор с параметрами. При создании s2 вызывается конструктор копирования.

18. Какие методы будут вызваны при выполнении следующих операторов:

```
student s1("Ivanov",20);
```

```
student s2;
```

```
s2 = s1;
```

Ответ: При создании s1 вызывается конструктор с параметрами. При создании s2 вызывается конструктор без параметров (по умолчанию). При присваивании s2 = s1 вызывается оператор присваивания (а не конструктор). Если оператор присваивания явно не определён, компилятор создаст его по умолчанию (побитовое копирование).

19. Какой конструктор будет использоваться при передаче параметра в функцию print():

```
void print(student a)
```

```
{ a.show(); }
```

Ответ: Будет использоваться конструктор копирования, так как параметр a передаётся по значению.

20. Класс описан следующим образом:

```
class Student
```

```
{
```

```
    string name;  
    int age;  
public:  
    void set_name(string);  
    void set_age(int);  
    ....  
};
```

Student p;

Каким образом можно присвоить новое значение атрибуту name объекта p?

Ответ: Через публичный метод set_name: p.set_name("новое имя"). Прямого доступа к атрибуту name нет, так как он объявлен в private.